Cours 09 — Stratégies de decoding

Constrained decoding · Teacher-forced scoring · Lexical-affinity bonus · Ordered-fallback rule

Focus code sur `src/generator.py`. Chaque concept est défini, motivé, puis illustré par un extrait du code du projet **callMeMaybe**.

Sommaire

1. Vue d'ensemble

1.1 Le problème : un petit LLM, du JSON fiable

1.2 Quatre idées, une seule pipeline

2. Constrained decoding

2.1 Définition

2.2 Comment c'est appliqué dans callMeMaybe

2.3 Exemples de code

3. Teacher-forced scoring

3.1 Intuition : scorer plutôt que générer

3.2 La log-vraisemblance, pas à pas

3.3 Log-sum-exp et stabilité numérique

3.4 Normalisation par longueur

3.5 Code : _select_function_name

4. Lexical-affinity bonus

4.1 Pourquoi un biais lexical ?

4.2 Mécanique : mots partagés, hors stopwords

4.3 Code : _lexical_affinity

4.4 Exemple pas-à-pas

5. Ordered-fallback rule

5.1 Le bug historique (a = b = 265)

5.2 Détection : hallucination et doublon

5.3 Récupération positionnelle

5.4 Code : _generate_param_value

6. Récapitulatif et bonnes pratiques

1. Vue d'ensemble

1.1 Le problème : un petit LLM, du JSON fiable

Qwen3-0.6B tient sur un MacBook mais n'est pas un GPT-4. Si on lui demande « écris du JSON », il réussit ~30 % du temps : virgules en trop, clés inventées, types incohérents. Le sujet exige pourtant **100 % de JSON valide** et des paramètres typés.

Le pari du projet : **ne jamais laisser le modèle décider librement**. Le JSON est assemblé côté Python ; le modèle n'intervient que sur des *choix bornés* (quelle fonction ? quel nombre ?) qu'on sait vérifier derrière.

1.2 Quatre idées, une seule pipeline

Les quatre techniques de ce cours travaillent ensemble :

Étape	Technique	Rôle
Cadre global	Constrained decoding	Borner chaque décision à un ensemble valide
Choix de fonction	Teacher-forced scoring	Classer les noms par log-vraisemblance
Tie-breaker	Lexical-affinity bonus	Injecter un signal déterministe quand le LLM hésite
Extraction de valeurs	Ordered-fallback rule	Rattraper hallucinations et doublons

> *« Le modèle est un conseiller, pas un décideur. Le Python tranche, le modèle pondère. »*

2. Constrained decoding

2.1 Définition

Constrained decoding : à chaque étape de génération, on restreint les tokens (ou les valeurs) que le modèle peut produire à un ensemble qui reste *valide* au regard d'une contrainte structurelle (JSON) et sémantique (schéma attendu).

Principe :

1. Le modèle produit des logits pour tous les tokens.
2. On identifie les tokens valides (qui préservent le schéma).
3. On met les logits des tokens invalides à $-\infty$.
4. On échantillonne uniquement parmi les tokens restants.

2.2 Comment c'est appliqué dans callMeMaybe

Dans ce projet, le constrained decoding est **implicite et structurel** : on n'édite pas les logits à chaque token, on restreint *l'espace de sortie* d'une étape à l'autre.

Décision	Ensemble borné
Nom de fonction	les noms présents dans functions_definition.json
Valeur d'un number	nombres extractibles par regex depuis le prompt
Valeur d'un boolean	{true, false} (comparaison directe des logits)
Valeur d'un string substitution	segments cités du prompt ou motifs connus
JSON final	validé par Pydantic avant sérialisation

2.3 Exemples de code

a) Le boolean : une décision en un logit

Le cas le plus pur de constrained decoding dans le projet. On ne laisse pas le modèle « écrire » true ou false, on compare directement les logits des deux tokens :

```
def _generate_boolean_value(
    model: Small_LLM_Model, prompt: str, param_name: str
) -> bool:
    q = (
        f"User request: {prompt}\n"
        f"Generate a boolean value for parameter '{param_name}'.\n"
        "Answer only true or false.\n"
        "Answer:"
    )
    ids = model.encode(q)[0].tolist()
    logits = np.array(
        model.get_logits_from_input_ids(ids), dtype=float
    )
    true_ids  = model.encode(" true")[0].tolist()  or \
                model.encode("true")[0].tolist()
    false_ids = model.encode(" false")[0].tolist() or \
                model.encode("false")[0].tolist()
    if not true_ids or not false_ids:
        return False
    return bool(logits[true_ids[0]] >= logits[false_ids[0]])
```

Note — L'ensemble valide a exactement deux éléments. On lit directement la coordonnée du logit correspondant au premier token de chaque mot, et on garde le plus grand. Aucune ambiguïté possible.

b) Le number : un automate qui filtre les tokens

Pour les nombres, on n'a pas un ensemble fini mais un **langage régulier**. On filtre token par token avec un petit automate :

```
for _ in range(16):
    logits = np.array(
        model.get_logits_from_input_ids(ids), dtype=float
    )
    next_id = int(np.argmax(logits))
    tok = model.decode([next_id])
    if not tok:
        break
    if not started:
        # Ignore leading whitespace until first meaningful char.
        if not tok.strip():
            ids.append(next_id); continue
        if not any(c.isdigit() or c in "-+." for c in tok):
            break   # drift -> fallback
        started = True
        out += tok; ids.append(next_id); continue
    # once started, only digits or dot are allowed
    if any(c.isdigit() or c == "." for c in tok) and "-" not in tok:
        out += tok; ids.append(next_id)
    else:
        break
```

États : *before-start* (ignore espace, attend signe/chiffre) → *reading* (accepte chiffres et point, refuse un 2^{e} signe). Tout ce qui sort de cette grammaire arrête la lecture : le JSON final ne pourra donc pas contenir autre chose qu'un nombre bien formé.

3. Teacher-forced scoring

3.1 Intuition : scorer plutôt que générer

Un modèle causal génère habituellement en *free decoding* : on lui demande la suite, il répond. Sur un 0.6B, le nom de fonction produit est très peu fiable : il peut écrire `add_numbers` au lieu de `fn_add_numbers`, inventer une fonction, ou choisir la mauvaise.

Le **teacher-forced scoring** renverse la perspective : au lieu de demander « quel nom ? », on **force** le modèle à considérer chaque candidat *déjà connu*, et on mesure à quel point ce candidat est probable dans le contexte.

> *« On ne demande pas au modèle de deviner, on lui demande d'évaluer. »*

3.2 La log-vraisemblance, pas à pas

Pour chaque fonction candidate, on calcule :

```
log P(name | prompt) = Σ_t log P(token_t | prompt, token_<t)
```

Algorithme (teacher forcing) :

1. Tokeniser le nom : name_ids = [t0, t1, ..., tn].
2. Partir du préambule (prompt + liste des fonctions).
3. Pour chaque token cible t_i : demander les logits, calculer log P(t_i | prefix)
4. Sommer les log-probabilités.

3.3 Log-sum-exp et stabilité numérique

Passer d'un logit à une log-probabilité demande une softmax :

```
log P(i) = logits[i] - log(Σ_j exp(logits[j]))
```

Problème : `exp(x)` déborde rapidement en float64 (au-delà de $x \approx 709$). Astuce **log-sum-exp** : on soustrait le maximum m avant d'exponentialiser :

```
m = max(logits)
logsumexp = m + log(Σ_j exp(logits[j] - m))
```

Mathématiquement identique, numériquement stable : le plus grand `exp(·)` vaut exactement `exp(0) = 1`, plus d'overflow.

3.4 Normalisation par longueur

Des noms plus courts ont mécaniquement une meilleure log-proba cumulée (moins de termes négatifs à sommer). Pour comparer des séquences de longueurs différentes, on **divise par le nombre de tokens** : on obtient la log-proba moyenne par token.

3.5 Code : _select_function_name

```
def _select_function_name(
    model: Small_LLM_Model,
    prompt: str,
    functions: list[FunctionDef],
) -> str:
    preamble = (
        "You must pick exactly one function name for the user request.\n"
        f"User request: {prompt}\n"
        "Available functions:\n"
```

```
        + "\n".join(f"- {f.name}: {f.description}" for f in functions)
        + "\nSelected function name:"
    )
    preamble_ids = model.encode(preamble)[0].tolist()
    best_name, best_score = None, -np.inf

    for func in functions:
        name_ids = model.encode(func.name)[0].tolist()
        if not name_ids: continue
        running_ids = preamble_ids.copy()
        score = 0.0
        for token_id in name_ids:
            logits = np.array(
                model.get_logits_from_input_ids(running_ids), dtype=float
            )
            m = float(np.max(logits))
            logsumexp = m + float(
                np.log(np.sum(np.exp(logits - m)))
            )
            score += float(logits[token_id] - logsumexp)
            running_ids.append(token_id)   # <-- teacher forcing
        score /= len(name_ids)             # <-- length normalisation
        score += _lexical_affinity(prompt, func)
        if score > best_score:
            best_score, best_name = score, func.name

    if best_name is None:
        raise ValueError("Unable to select function name")
    return best_name
```

Note — Trois choses à repérer dans ce code : **(1)** l'ajout du token cible au prefix à chaque itération (teacher forcing), **(2)** la soustraction du max avant `exp` (log-sum-exp), **(3)** la division par la longueur du nom avant la comparaison.

4. Lexical-affinity bonus

4.1 Pourquoi un biais lexical ?

Sur un prompt court comme « Sum of -7 and 8 », le score LLM entre `fn_add_numbers` et `fn_substitute_string_with_regex` peut être *quasi-égal*. Un bruit d'échantillonnage suffit alors à faire gagner la mauvaise fonction.

Le modèle a beau être de l'IA, le mot « sum » dans le prompt **est** un signal. Un humain qui choisirait verrait immédiatement l'overlap « sum » → description de `fn_add_numbers`. On injecte ce signal comme un petit bonus déterministe.

> *« Cumuler un signal LLM (souple) et un signal lexical (rigide) : principe général en NLP pour stabiliser un classement instable. »*

4.2 Mécanique : mots partagés, hors stopwords

La fonction compte les mots du **nom** et de la **description** qui apparaissent dans le prompt, pondérés à +0.5 chacun. Garde-fous :

- Les mots trop courts (< 3 lettres) sont ignorés.
- Les stopwords (« the », « of », « and », « fn »…) sont filtrés.
- La comparaison se fait sur des mots entiers (\b...\b) pour éviter que « add » match
- On opère en lowercase des deux côtés.

4.3 Code : _lexical_affinity

```
_STOPWORDS = {
    "the", "a", "an", "of", "to", "and", "or", "for", "is", "are", "be",
    "in", "on", "at", "by", "with", "as", "it", "this", "that", "these",
    "those", "their", "them", "two", "fn", "return", "returns", "result",
    "value", "function", "call", "input", "output",
}

def _lexical_affinity(prompt: str, func: FunctionDef) -> float:
    """Score overlap between prompt and function metadata."""
    lowered = prompt.lower()
    name_words = set(re.findall(r"[a-z]+", func.name.lower()))
    desc_words = set(re.findall(r"[a-z]+", func.description.lower()))
    words = (name_words | desc_words) - _STOPWORDS
    bonus = 0.0
    for w in words:
        if len(w) >= 3 and re.search(rf"\b{re.escape(w)}\b", lowered):
            bonus += 0.5
    return bonus
```

Note — Le bonus est ajouté *après* la normalisation du score LLM. +0.5 par mot matché, c'est calibré pour peser dans les décisions serrées sans écraser les scores LLM clairs.

4.4 Exemple pas-à-pas

Prompt : `"Sum of -7 and 8"`.

Fonction	Mots extraits (hors stopwords)	Matches	Bonus
fn_add_numbers	add, numbers, sum, together, their	sum	+0.5
fn_greet	greet, generate, greeting, message, person, name	–	0.0
fn_substitute_string_with_regex	substitute, string, regex, replace, occurrences, matching, pattern	–	0.0

Avec des scores LLM très proches, le +0.5 suffit à faire gagner `fn_add_numbers`. Le prompt restait décidable par le LLM seul sur des cas simples, mais le bonus verrouille le comportement sur les cas limites.

5. Ordered-fallback rule

5.1 Le bug historique (a = b = 265)

Avant la règle, sur le prompt "`What is the sum of 265 and 345?`", la sortie était :

```
{"a": 265.0, "b": 265.0}   # BUG: b devrait valoir 345.0
```

Pourquoi ? Le prompt que le générateur passe au modèle pour b est quasiment identique à celui pour a (on change juste le nom du paramètre). Un petit LLM renvoie donc souvent la même valeur (le premier nombre du prompt) pour les deux paramètres.

5.2 Détection : hallucination et doublon

On déclenche le fallback dans deux cas :

Cas	Condition	Exemple
Hallucination	la valeur générée n'existe pas dans le prompt	prompt « sum of 2 and 3 », modèle produit 42
Doublon	la valeur est déjà utilisée par un paramètre précédent	prompt « sum of 265 and 345 », le modèle répète 265 (déjà pris par

5.3 Récupération positionnelle

Quand le fallback se déclenche, on ne laisse pas le hasard : le i-ième paramètre prend le i-ième nombre extrait du prompt, dans l'ordre d'apparition.

```
def _fallback_number_value(
    prompt: str, param_name: str, param_index: int = 0
) -> float:
    """Return a deterministic number from prompt values."""
    nums = _extract_numbers_from_prompt(prompt)
    if not nums:
        return 0.0
    if param_index < len(nums):
        return nums[param_index]       # <-- récupération positionnelle
    # Legacy mapping kept for compatibility.
    if param_name == "b" and len(nums) >= 2:
        return nums[1]
    return nums[0]
```

L'extraction utilise une regex simple qui capture signe et décimales :

```
def _extract_numbers_from_prompt(prompt: str) -> list[float]:
    return [float(x) for x in re.findall(r"-?\d+(?:\.\d+)?", prompt)]
```

5.4 Code : _generate_param_value

La règle est branchée dans le dispatcher typé :

```
def _generate_param_value(
    model: Small_LLM_Model,
    prompt: str,
    param_name: str,
    param_type: str,
    param_index: int = 0,
    already_used: list[Any] | None = None,
) -> Any:
    used = already_used or []
    if param_type == "number":
        generated = _generate_number_value(model, prompt, param_name)
        nums_in_prompt = _extract_numbers_from_prompt(prompt)
        # Fall back when generation hallucinates or duplicates values.
```

```
        needs_fallback = bool(nums_in_prompt) and (
            generated not in nums_in_prompt
            or (generated in used
                and len(set(nums_in_prompt)) > 1)
        )
        if needs_fallback:
            return _fallback_number_value(
                prompt, param_name, param_index
            )
        return generated
    ...
```

Et l'appelant (`generate_function_call`) passe l'index et les valeurs déjà choisies :

```
for idx, (pname, pdef) in enumerate(
    ctx.chosen_func.parameters.items()
):
    params[pname] = _generate_param_value(
        model,
        prompt,
        pname,
        pdef.type,
        param_index=idx,                # <-- i-ième paramètre
        already_used=list(params.values()),  # <-- valeurs déjà choisies
    )
```

Note — Nuance subtile dans la condition doublon : `len(set(nums_in_prompt)) > 1`. On ne bascule en fallback que si le prompt contient au moins deux nombres *distincts*. Sinon (prompt « add 5 and 5 »), le doublon est légitime et on garde la valeur générée.

6. Récapitulatif et bonnes pratiques

6.1 Ce que chaque technique apporte

Technique	Ce qu'elle garantit
Constrained decoding	le JSON produit est 100 % syntaxiquement et sémantiquement valide
Teacher-forced scoring	la fonction choisie est toujours prise dans un ensemble connu, classée par un critère stable
Lexical-affinity bonus	robustesse face aux scores LLM quasi-égaux, sur un petit modèle
Ordered-fallback rule	rattrape les hallucinations et doublons sans jamais renvoyer de valeur inventée

6.2 Le bon réflexe

> *« Le modèle est une boîte à propositions. Le code est la boîte à décisions. Plus le modèle est petit, plus on déplace de logique côté code — sans jamais violer le sujet : le LLM reste le classifieur principal des fonctions, les garde-fous n'agissent qu'en aval ou en tie-break. »*

6.3 À retenir pour la soutenance

1. Le JSON valide n'est pas produit par le modèle mais assemblé côté Python.
2. Scorer plutôt que générer : le teacher forcing permet au modèle de classer des candidats connus.
3. log-sum-exp est la recette standard pour stabiliser une softmax numérique.
4. Un petit bonus déterministe (lexical) suffit à verrouiller une décision que le LLM rendait ambigüe.
5. Chaque fallback est motivé par un signal concret : hallucination (valeur absente du prompt) ou doublon (valeur déjà

> *« Un petit LLM bien encadré se comporte comme un grand LLM mal encadré. Le projet callMeMaybe est une étude de cas de cet encadrement. »*